

# Advanced Machine Learning

## Lab 2 --- Module 1

### Instructions on how to submit your work on Canvas

To convert your Notebook to an HTML file, follow these steps:

- Open the Notebook that you want to convert to an HTML file.
- Ensure that the notebook is saved with the latest changes.
- In the Notebook interface, navigate to File in the menu bar.
- Click on Download as and select HTML (.html) from the dropdown menu. This will initiate the conversion process.
- The Jupyter Notebook will be converted to an HTML file and automatically downloaded to the default download location on your computer.
- Locate the downloaded HTML file on your computer and verify that the conversion was successful.
- Optionally, open the HTML file in a web browser to view and verify the content.
- Upload your HTML file on Canvas.

Make sure to save your Notebook before attempting to convert it to an HTML file.

### Exercise 1 (1 point)

Briefly explain what are the differences between stochastic gradient descent (SGD), mini-batch gradient descent, and batch gradient descent.

### Exercise 2 (2 points)

Generate linear-looking data using the given Python file provided to you.

1. Implement gradient descent for the MSE cost function with a fixed learning rate and 1000 number of iterations. Try three different learning rates 0.02, 0.1, 0.5 and show the effect of the learning rate in the convergence behavior of the algorithm. For this, you can plot the first 10 steps of gradient descent for each of the learning rates. Which learning rate provides the fastest convergence?

2. Implement stochastic gradient descent. Try different learning schedule hyper-parameters, and a random initialization. Show the first 20 steps of stochastic gradient descent. What do you observe?
3. Implement mini-batch gradient descent with mini-batch size equal to 20 and plot the first 20 steps of mini-batch gradient descent.
4. Compare stochastic gradient descent, gradient descent, and mini-batch gradient descent in terms of the paths taken by the three gradient descent algorithms in the parameter space during training. What do you observe? Which algorithm shows faster convergence?

### Exercise 3 (2 points)

Apply Newton's method to

$$J(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^\top \mathbf{H} \boldsymbol{\theta} + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2 \quad (1)$$

starting from a random point of  $\boldsymbol{\theta}$ . Use

$$\mathbf{H} = \begin{bmatrix} 2 & 0 \\ 0 & 2000 \end{bmatrix}, \quad \lambda = 10 \quad (2)$$

What do you observe in terms of the number of iterations needed for the algorithm to converge? Next, apply gradient descent to the same optimization problem. Which algorithm is faster in terms of the number of iterations needed for convergence?

**Hint:** For the implementation and details of Newton's method, use the following link:  
[https://xavierbourretsicotte.github.io/Intro\\_optimization.html#Newton-Raphson%27s-method](https://xavierbourretsicotte.github.io/Intro_optimization.html#Newton-Raphson%27s-method)